(ES-452: Major Project - Dissertation)
SOFTWARE REQUIREMENT SPECIFICATION ON

ARCHITECTURE AND DEVELOPMENT OF AN ENTERPRISE-GRADE AI CHATBOT WITH SEMANTIC KNOWLEDGE RETRIEVAL

submitted in partial fulfillment of the requirement
for the award of the degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE & ENGINEERING

Submitted By:

UJJWAL KUMAR

(Enrollment No: 00818007223)

Under the supervision of:

Mr. UPENDRA PRATAP PANDEY
(ASSISTANT PROFESSOR)

Department of Computer Science & Engineering
Delhi Technical Campus
28/1, Knowledge Park-III, Greater Noida - 201306 (U.P.)

MAY 2026

Software Requirement Specification

1. Introduction:

The Need for Semantic Intelligence
As a final-year Computer Science student, I've spent a lot of time looking at how organizations handle data. In the modern corporate world, the problem isn't a lack of information; it's the inability to find it. Most critical knowledge is trapped in "dead" formats like PDF repositories, project logs, and technical manuals. Standard keyword searches fail because they don't understand the intent behind a query. If I search for "compensation" but the document says "reimbursement," a traditional system gives me nothing [cite: 41, 145]. Furthermore, while Large Language Models (LLMs) like ChatGPT are incredible, you can't just feed them sensitive enterprise data. There are massive privacy risks and the constant threat of "hallucinations"—where the AI confidently makes up facts when it doesn't know the answer [cite: 42, 113, 149]. This project was born out of a need to build a secure, "interactive corporate brain" that stays entirely within a private network while providing fact-grounded responses [cite: 58, 115].

2. Problem Definition and Motivation

The primary challenge I addressed was the "Crisis of Unstructured Data" [cite: 138]. For my major project, I wanted to move beyond academic theory and create something production-ready. The goal was to solve four main pain points: the failure of traditional search, the black-box nature of public AI, data sovereignty concerns, and the sheer complexity of deploying AI

in a corporate setting [cite: 71-75, 137-164]. By using a Retrieval-Augmented Generation (RAG) framework, we can ensure the AI only answers based on verified internal documents, effectively eliminating hallucinations [cite: 44, 232].

Key Objective: To develop a locally-hosted AI solution that ensures 100% data privacy while providing ultra-low latency semantic retrieval [cite: 57, 131, 377].

3. The Technical Stack: Why We Chose It

Building an enterprise-grade tool on a student budget (and hardware) required a very strategic selection of open-source tools. We opted for a completely local infrastructure [cite: 216].

Llama 3.2 via Ollama: This serves as the "brain." Llama 3.2 is lightweight enough to run on consumer hardware but powerful enough for complex reasoning [cite: 51, 169, 371].

ChromaDB: This is our AI-native vector database. It allows us to perform mathematical "similarity" searches rather than just word matching [cite: 50, 192, 337].

FastAPI: Chosen for the backend because of its high performance and native support for asynchronous tasks, which is vital when processing large PDFs [cite: 54, 191, 333].

Glassmorphism UI: We used modern web principles (HTML5, CSS3, ES6) to create a premium, intuitive frontend that feels like a professional enterprise tool [cite: 55, 124, 194].

4. Proposed Methodology and Architecture

The architecture is built on a decoupled RAG framework [cite: 211]. We split the system into three layers: Data Ingestion, Vector Storage, and Inference [cite: 213].

4.1. The Data Pipeline

When a user uploads a PDF, the pypdf library extracts the raw text [cite: 223]. I implemented a "Recursive Character Splitting" strategy, breaking the text into 1000-character chunks with a 100-character (10%) overlap [cite: 238]. This overlap is crucial; it ensures that if a sentence is cut in half at the end of a chunk, the context isn't lost [cite: 239]. These chunks are then converted into 768-dimension numerical vectors using an embedding model [cite: 48, 122, 240].

4.2. Dual-Mode Logic

One unique feature I'm proud of is the "Dual-Mode" logic [cite: 53, 200, 259]. When a user asks a question, the system first calculates the Cosine Similarity distance to find the most relevant document segments [cite: 50, 199, 256]. If relevant context is found, it uses RAG mode. If no document context exists, the system automatically falls back to the model’s general knowledge [cite: 53, 387].

5. Implementation and Hardware Feasibility

A big part of my feasibility study was ensuring this could run without a supercomputer. We found that local models like Llama 3.2 run efficiently on hardware with 8GB of RAM, though 16GB and a dedicated GPU (like an RTX 3060) are recommended for sub-2-second response times [cite: 129, 134, 179-185, 398].
I also developed a "one-click" deployment script (run_chatbot.bat) to automate the environment setup, making it accessible even for people without a technical background [cite: 201, 264, 305].

6. Testing and Performance Metrics

We didn't just build it; we stressed it. Our multi-layered testing strategy focused on "Recall Rate" and "Inference Latency" [cite: 271-272]. We achieved 98% text recall for relevant entities and a "Time to First Token" of under 2 seconds on the recommended hardware [cite: 205, 291]. Most importantly, we conducted a "Data Sovereignty Audit" by monitoring network traffic to confirm that absolutely no data was leaked to external APIs—fulfilling our primary security objective [cite: 285].

7. Limitations and Future Scope

Being honest about limitations is part of good engineering. Currently, the system is optimized for text-based PDFs and lacks an OCR layer for scanned images [cite: 314-315]. It also heavily depends on the local GPU for speed [cite: 311]. Moving forward, I plan to integrate Multimodal Support so the chatbot can interpret charts and diagrams, and Enterprise Authentication (OAuth2) for production environments [cite: 320-323].
8. Conclusion

This project demonstrates that private, local AI is not just a dream—it's a practical, cost-effective reality for modern organizations [cite: 300]. By combining modern CSE principles

with open-source models, we've turned static archives into a living, breathing corporate brain [cite: 58, 307].